

EDA

Ayden Kelleher, Daniel Havens, Douglas Roth

```
library(tidyverse)
library(ggplot2)
library(dplyr)
library(corrplot)
```

Data Summary

Our data is from the college scorecard data, [here](#)

We are looking at the most recent field of study dataset. This consists of 229188 observations with 174 columns, regarding students in the 2019/2020 school year.

Our institutional dataset is also pretty big, with 6694 rows and 3306 columns. We will join these dataframes on the university id, after reducing them in size.

This is how we loaded our data in.

```
data <- read_csv("Most-Recent-Cohorts-Field-of-Study.csv") #on individuals
institution_data <- read_csv("MERGED2019_20_PP.csv") #on institutions
```

We then joined using left join.

```
joined_df <- institution_data %>%
  left_join(data, by = "OPEID6")
```

This gave us a monstrous dataframe, with 3500 columns, and over 200000 rows. To reduce our data to a more manageable size, we did the following.

```
df_clean <- joined_df[, colMeans(is.na(joined_df)) <= 0.50]
df_clean <- df_clean %>%
  select(where(~ n_distinct(.) > 1))

threshold <- 0.01

df <- df_clean %>%
  select(where(~ {
    if (!is.numeric(.x)) {
      TRUE
    } else {
      v <- var(.x, na.rm = TRUE)
      !is.na(v) && v >= threshold
    }
  })))
```

This still had over 2000 columns. So what we did was we chose X many predictors that we thought could be interesting, to investigate further.

```
interesting_col_names <- c(
  "EARN_MDN_4YR",
  "OPEID6",
  "CITY",
  "SCH_DEG",
  "MAIN.x",
  "NUMBRANCH",
  "PREDEG",
  "HIGHDEG",
  "ST_FIPS",
  "REGION",
  "ADM_RATE_ALL",
  "PCIP12",
  "PCIP24",
  "PCIP51",
  "PCIP52",
  "UGDS",
  "UG25ABV",
  "UGDS_WHITE",
  "UGDS_BLACK",
  "UGDS_HISP",
  "PPTUG_EF",
  "COSTT4_A",
```

```

"TUITIONFEE_IN",
"TUITIONFEE_OUT",
"TUITFTE",
"INEXPFTE",
"AVGFACSAL",
"PFTFAC",
"C150_4",
"PFTFTUG1_EF",
"D150_4",
"C150_4_WHITE",
"C150_4_BLACK",
"C150_4_HISP",
"DEATH_YR8_RT",
"COMP_ORIG_YR8_RT",
"COMP_4YR_TRANS_YR8_RT",
"PELL_COMP_4YR_TRANS_YR8_RT",
"PELL_WDRAW_ORIG_YR8_RT",
"NOPELL_WDRAW_ORIG_YR8_RT",
"NOT1STGEN_COMP_4YR_TRANS_YR2_RT",
"FIRSTGEN_COMP_ORIG_YR2_RT",
"TUITFTE",
"OVERALL_YR2_N",
"DEBT_MDN",
"CREDESC",
"DEBT_ALL_STGP_ANY_MEAN",
"BBRR1_FED_UG_DFLT",
"DISTANCE",
"CONTROL.x",
"CONTROL.y",
"SATVRMID",
"SATMTMID"
)

```

We then created a dataframe with those interesting attributes. However, we were think about using career earnings as an outcome, thus we only are interested on results with a numeric earnings. Thus we get rid of all rows that have a character or na value in `EARN_MDN_4YR` column, or career earnings after 4 years.

But we still had the issue that there were many different columns with many NA values, some close to 40%. For simplicity, we decided to drop NA values. This has the potential to introduce bias in our data, because there could be some other factor influencing career earnings and whether or not there are many NA values.

```
very_trimmed_df <- df[interesting_col_names]
```

```
final_df <- very_trimmed_df[!is.na(as.numeric(as.character(very_trimmed_df$EARN_MDN_4YR))), ]
```

```
na_df <- na.omit(final_df)
```

All of this was done on a different document. Loading in the result from that process gives us the following dataset.

```
df <- read.csv("final_data.csv")
```

We still have some decisions to make regarding what data to use. Much of the variables we have are 75% or more just PS, which likely means primarily suppressed, which means not included since the sample size was too small, or the data was private. Since many of the predictors consist of things like debt after college, it makes sense that it is disclosed to the public (people like us).

EDA Begins

```
df %>% select(is.numeric, - OPEID6) %>% #only continuous output, without passenger id
cor(use = "pairwise.complete.obs") %>%
corrplot(type='lower',
          method = "color",      # Use colored squares/cells for compactness
          order = "hclust",      # Cluster similar variables together
          tl.cex = 0.5,          # Shrink text label size (adjust as needed)
          tl.col = "black",      # Text label color
          tl.srt = 45,           # Rotate text labels 45 degrees
          diag = FALSE)          # Hide self-correlations on the diagonal
```

Warning: Use of bare predicate functions was deprecated in tidysselect 1.1.0.

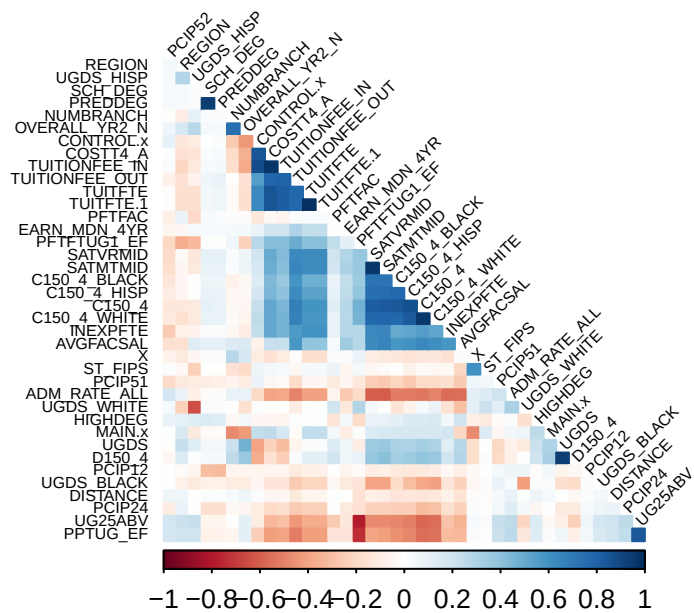
i Please use wrap predicates in `where()` instead.

Was:

```
data %>% select(is.numeric)
```

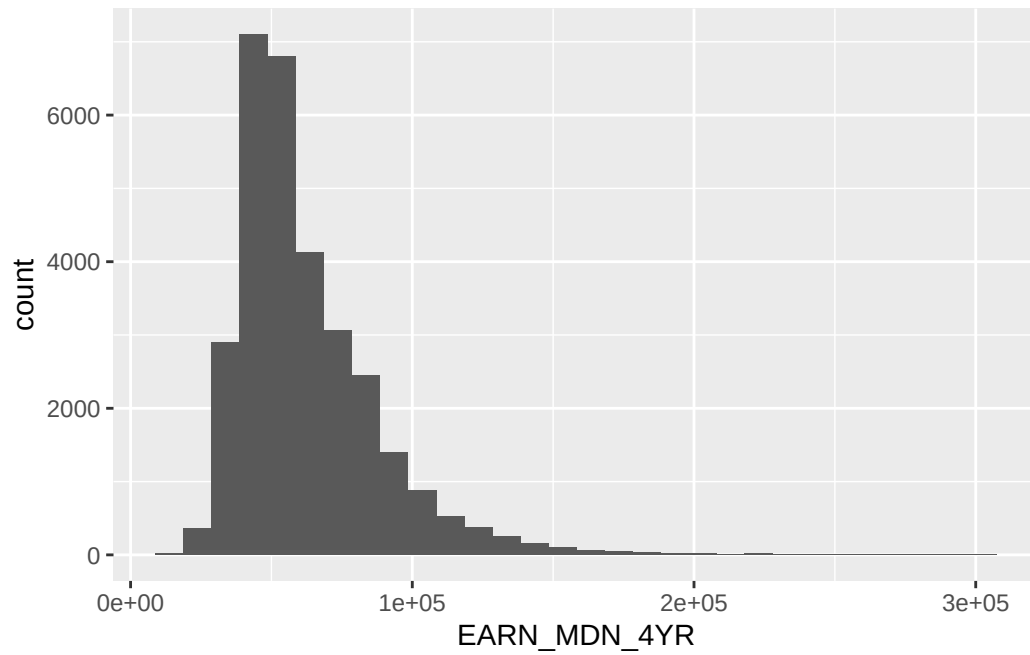
Now:

```
data %>% select(where(is.numeric))
```



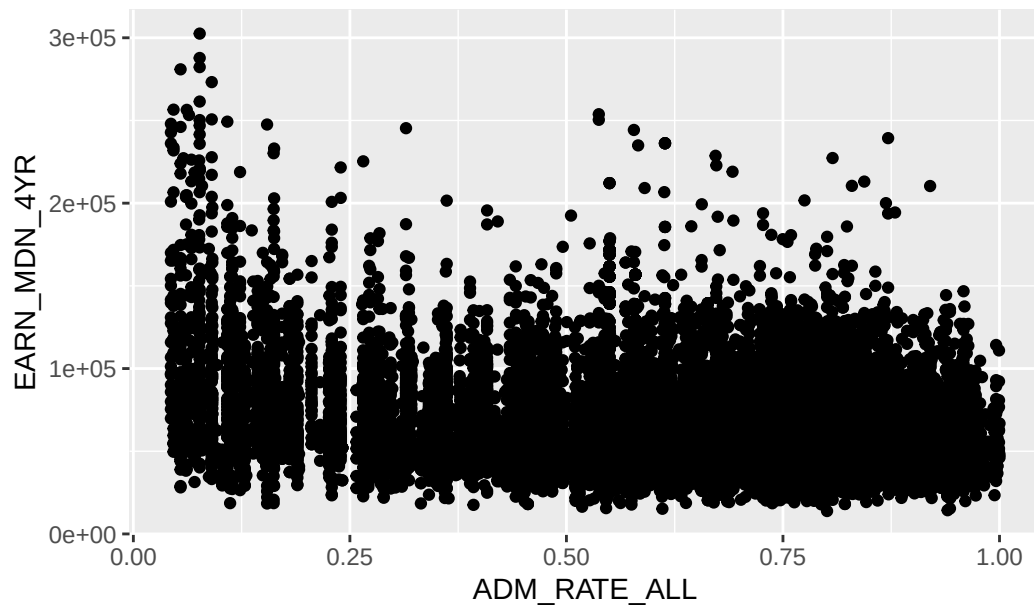
```
ggplot(df, aes(x = EARN_MDN_4YR)) +  
  geom_histogram()
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



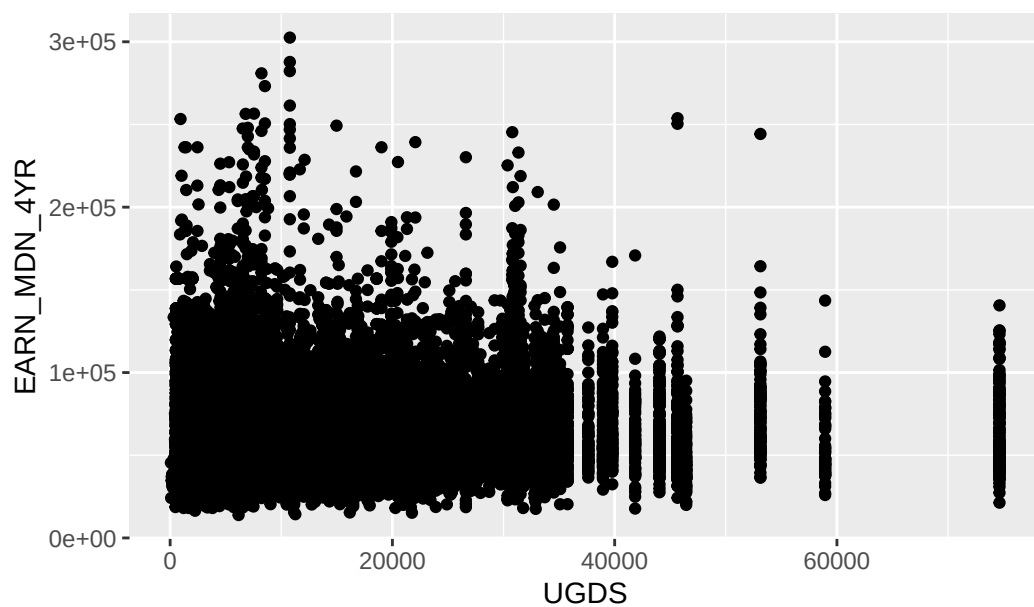
```
ggplot(df, aes(x=ADM_RATE_ALL, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "Admission Rate versus Earning after 4 years"  
  )
```

Admission Rate versus Earning after 4 years



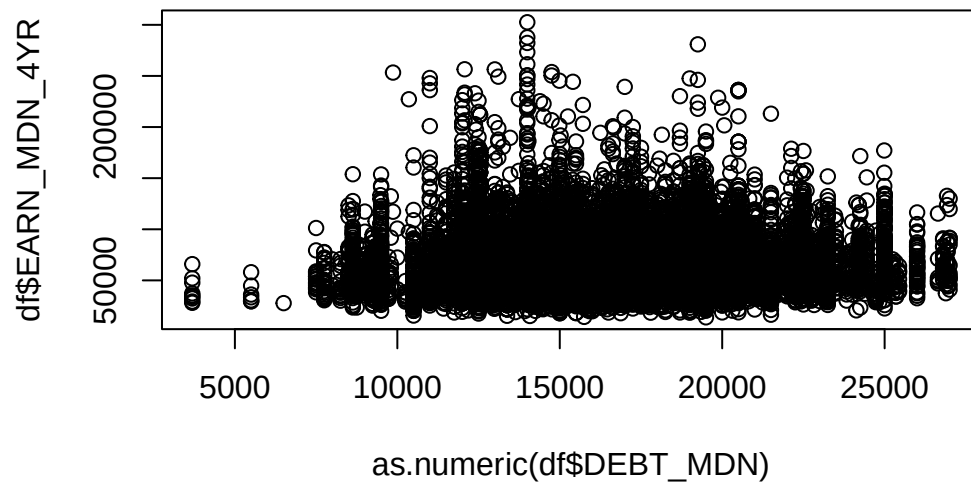
```
ggplot(df, aes(x=UGDS, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "Enrollment of undergraduate certificate/degree-seeking students versus Earning a  
  )
```

Enrollment of undergraduate certificate/degree-seeking stud



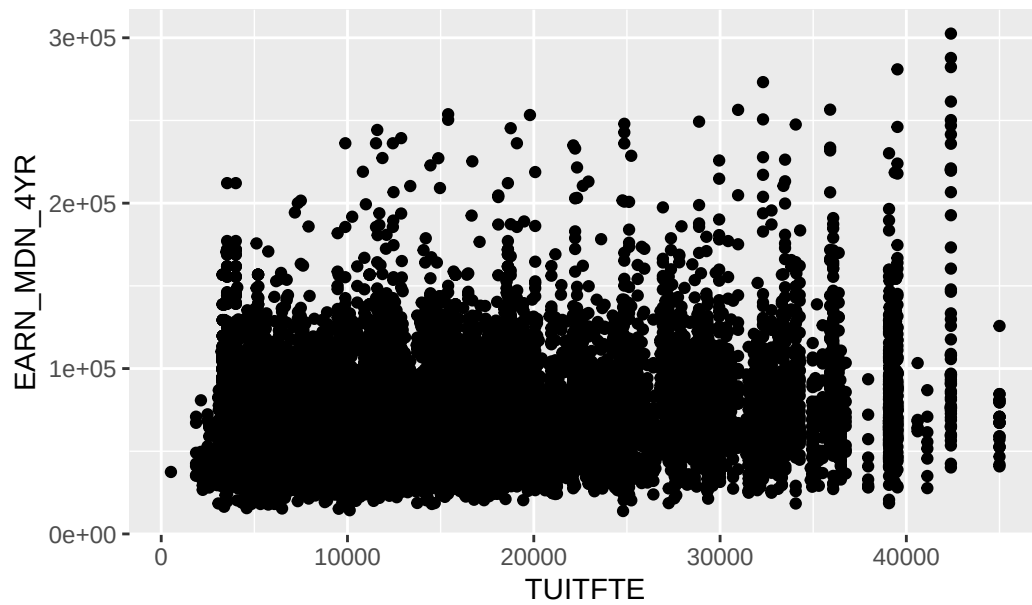
```
plot(as.numeric(df$DEBT_MDN), y=df$EARN_MDN_4YR)
```

Warning in plot(as.numeric(df\$DEBT_MDN), y = df\$EARN_MDN_4YR): NAs introduced by coercion

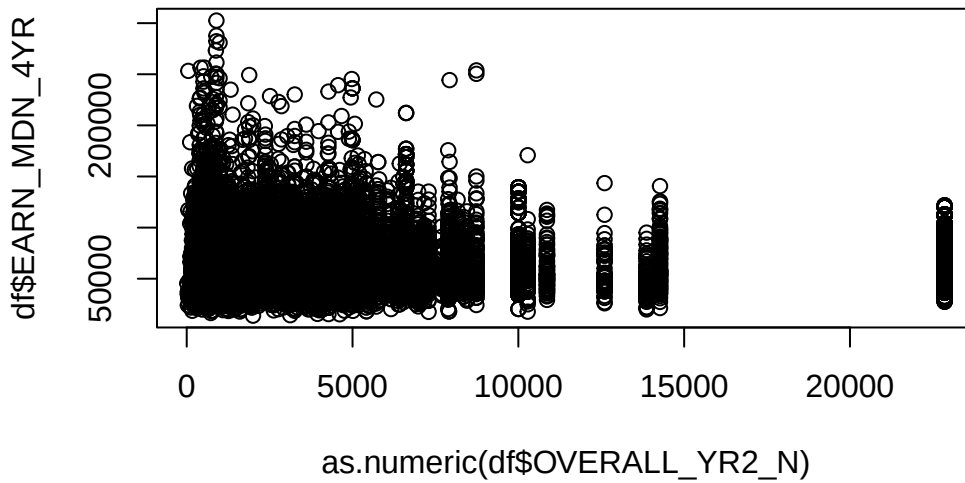


```
ggplot(df, aes(x=TUITFTE, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "Tuition fee per student versus Earning after 4 years"  
  )
```

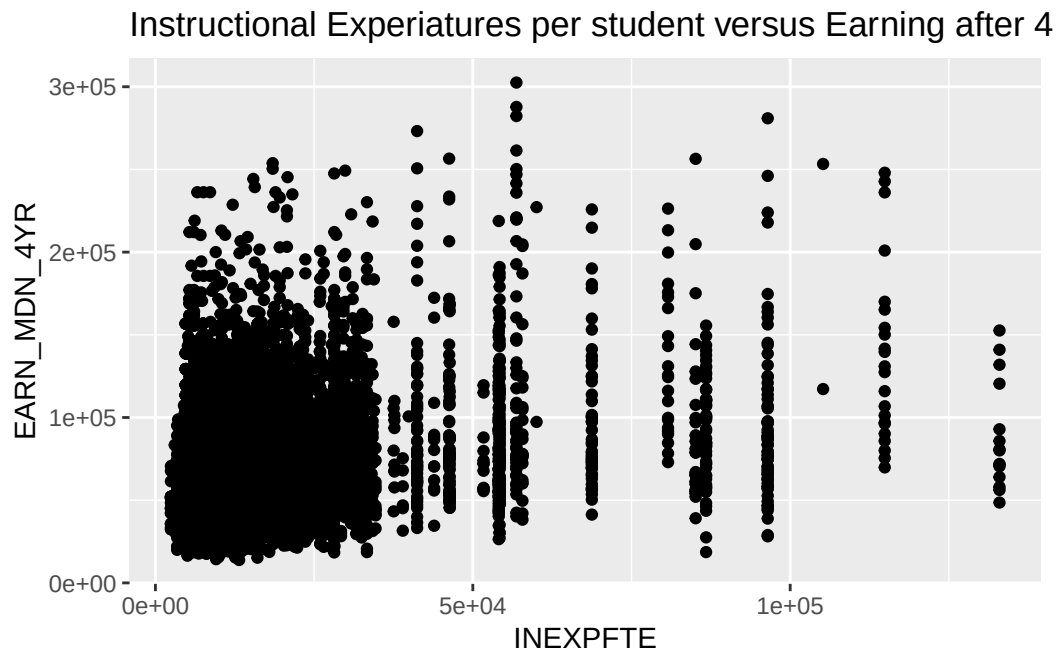
Tuition fee per student versus Earning after 4 years



```
plot(as.numeric(df$OVERALL_YR2_N), df$EARN_MDN_4YR)
```

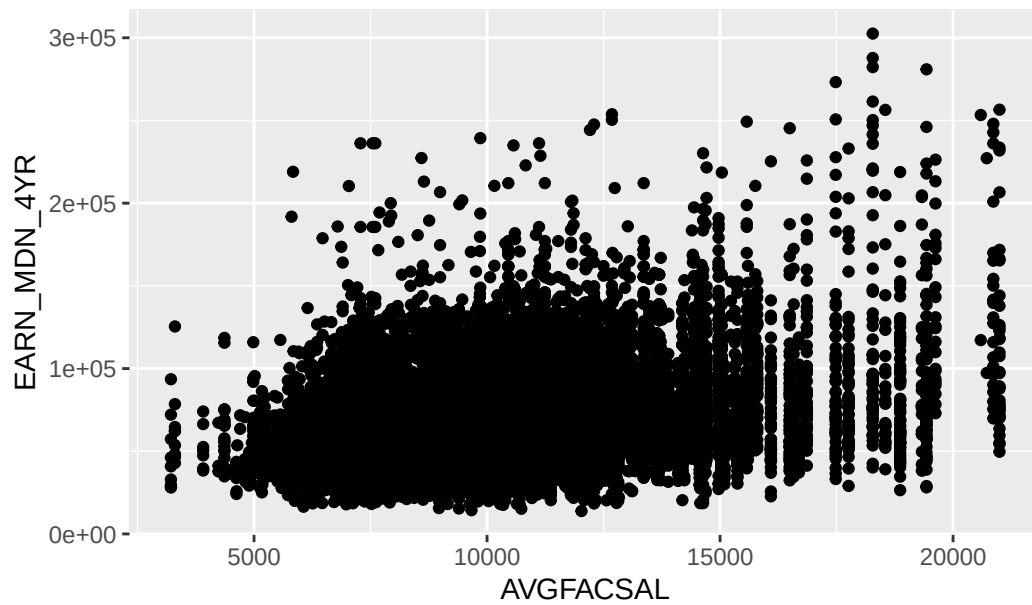


```
ggplot(df, aes(x=INEXPFTE, y = EARN_MDN_4YR)) +
  geom_point()+
  labs(
    title = "Instructional Experiatures per student versus Earning after 4 years"
  )
```



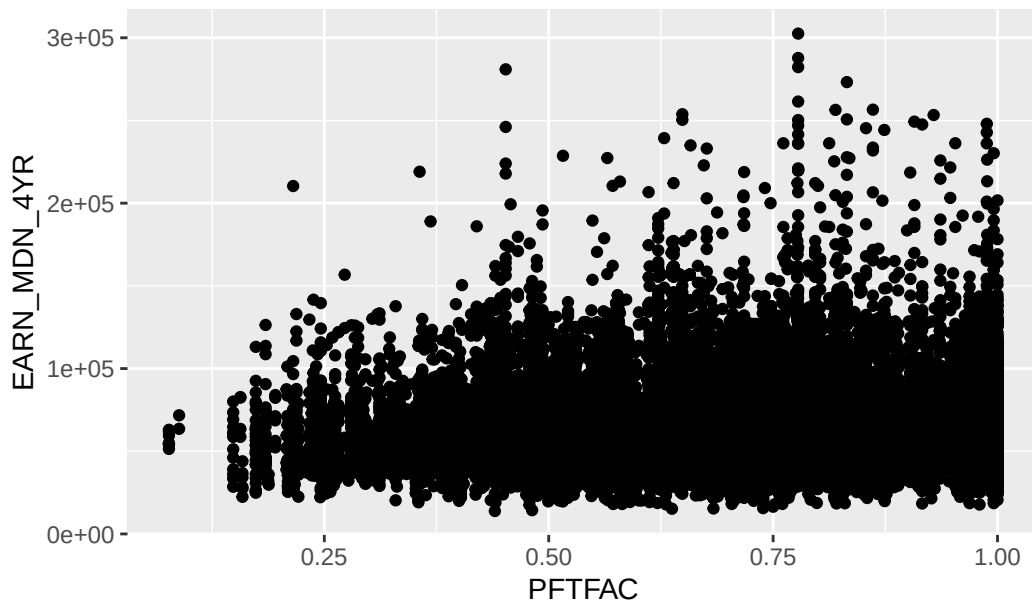
```
ggplot(df, aes(x=AVGFACSAL, y = EARN_MDN_4YR)) +
  geom_point()+
  labs(
    title = "Average Faculty Salary versus Earning after 4 years"
  )
```

Average Faculty Salary versus Earning after 4 years



```
ggplot(df, aes(x=PFTFAC, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "Percent of Faculty Full Time versus Earning after 4 years"  
  )
```

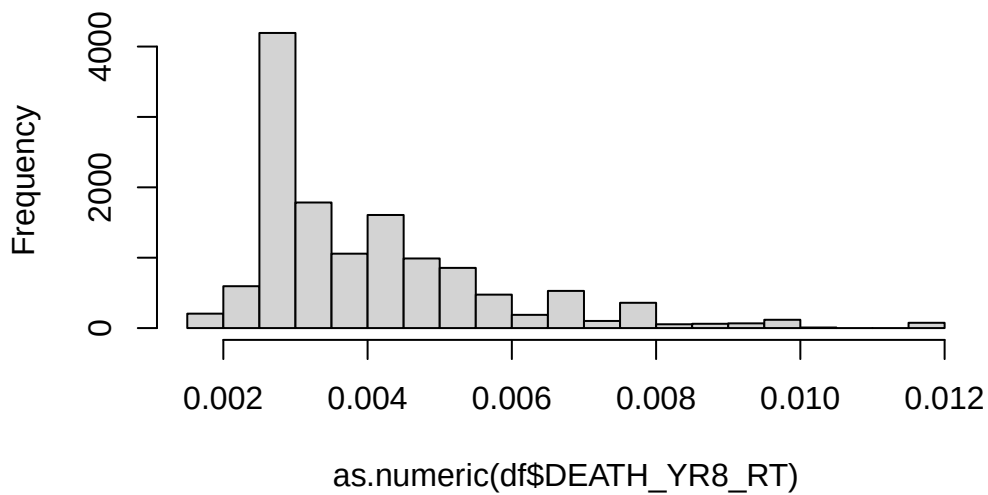
Percent of Faculty Full Time versus Earning after 4 years



```
hist(as.numeric(df$DEATH_YR8_RT)) #need to convert
```

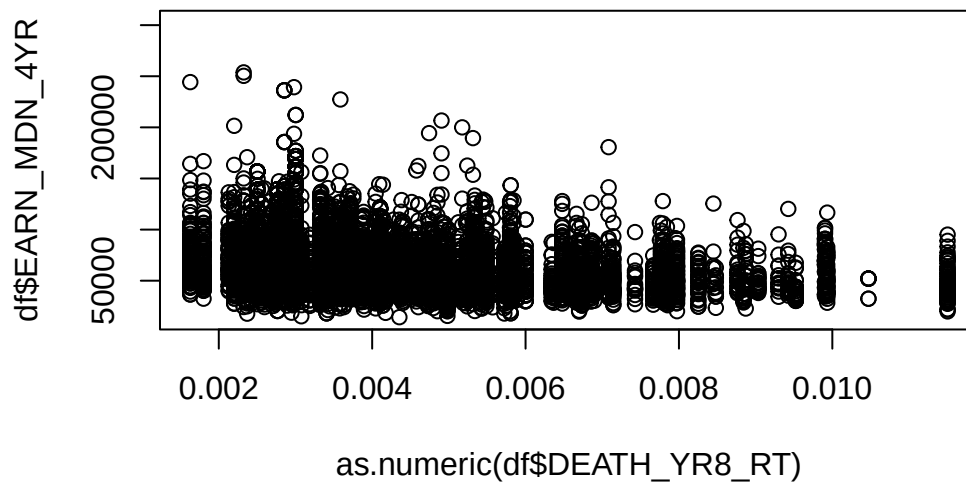
Warning in hist(as.numeric(df\$DEATH_YR8_RT)): NAs introduced by coercion

Histogram of as.numeric(df\$DEATH_YR8_RT)



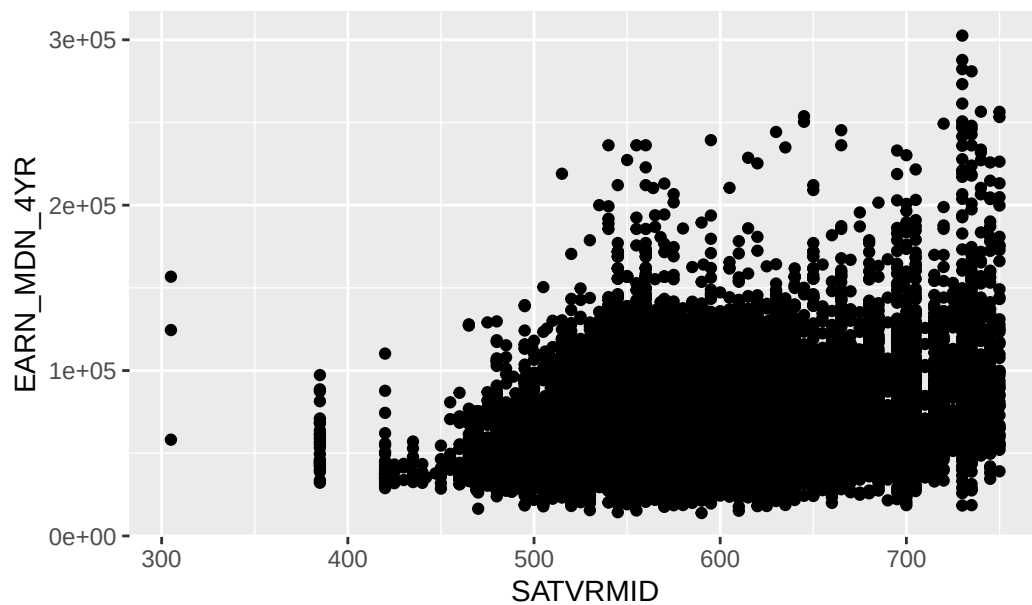
```
plot(as.numeric(df$DEATH_YR8_RT), df$EARN_MDN_4YR)
```

Warning in plot(as.numeric(df\$DEATH_YR8_RT), df\$EARN_MDN_4YR): NAs introduced by coercion

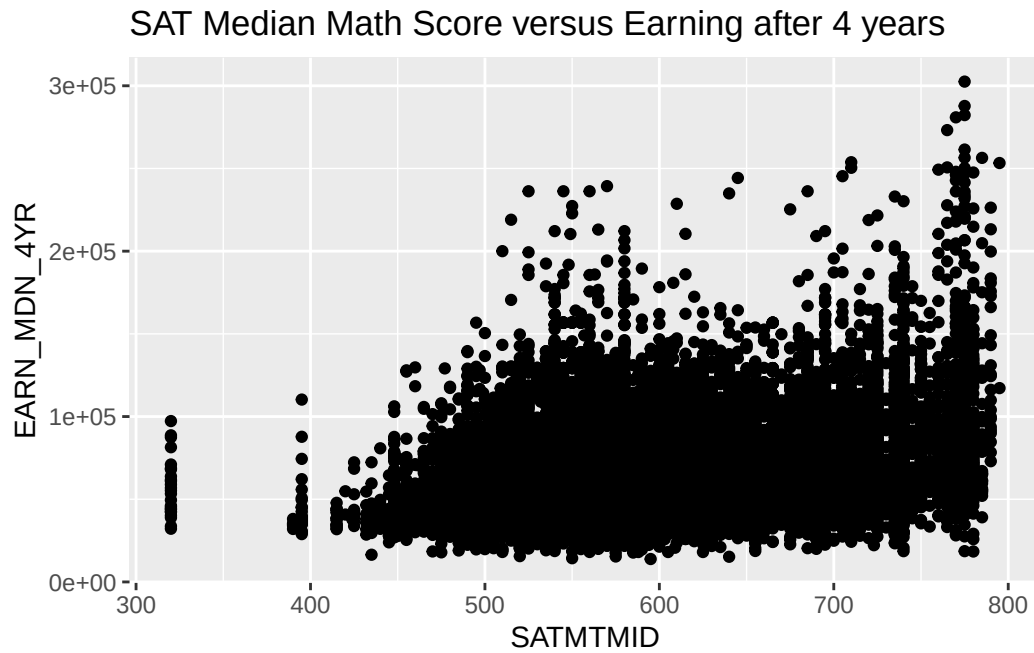


```
ggplot(df, aes(x=SATVRMID, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "SAT Median Critical Reading Score versus Earning after 4 years"  
  )
```

SAT Median Critical Reading Score versus Earning after 4 years

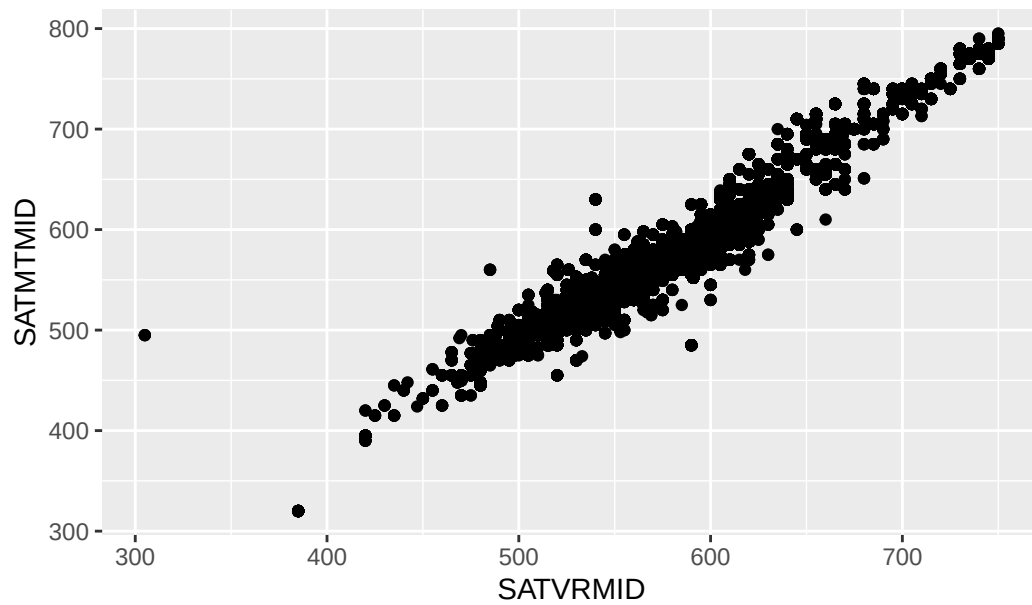


```
ggplot(df, aes(x=SATMTMID, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "SAT Median Math Score versus Earning after 4 years"  
  )
```



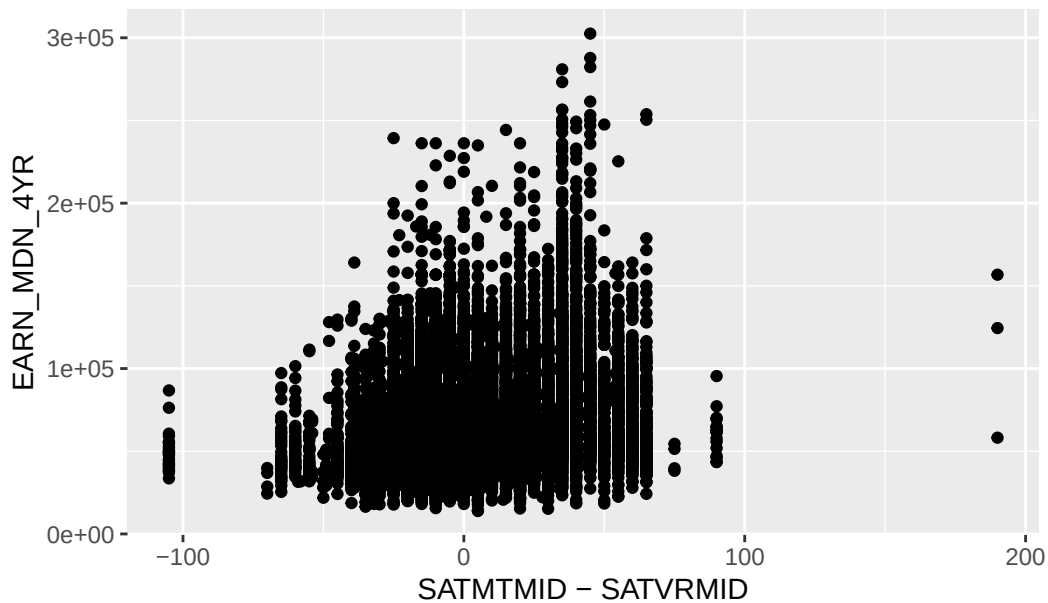
```
ggplot(df, aes(x=SATVRMID, y = SATMTMID)) +  
  geom_point()+  
  labs(  
    title = "SAT Median Critical Reading Score versus SAT Median Math Score"  
  )
```

SAT Median Critical Reading Score versus SAT Median Math Score



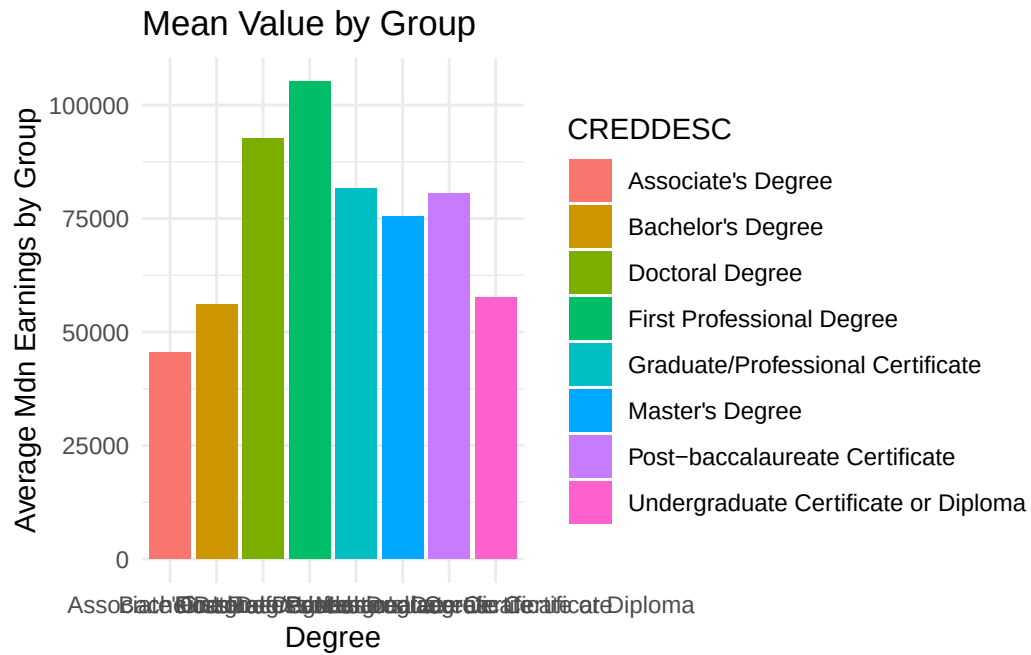
```
ggplot(df, aes(x=SATMTMID - SATVRMID, y = EARN_MDN_4YR)) +  
  geom_point()+  
  labs(  
    title = "SAT Median Math Minus SAT Median Critical Thinking versus Earnings"  
  )
```

SAT Median Math Minus SAT Median Critical Thinking versu:

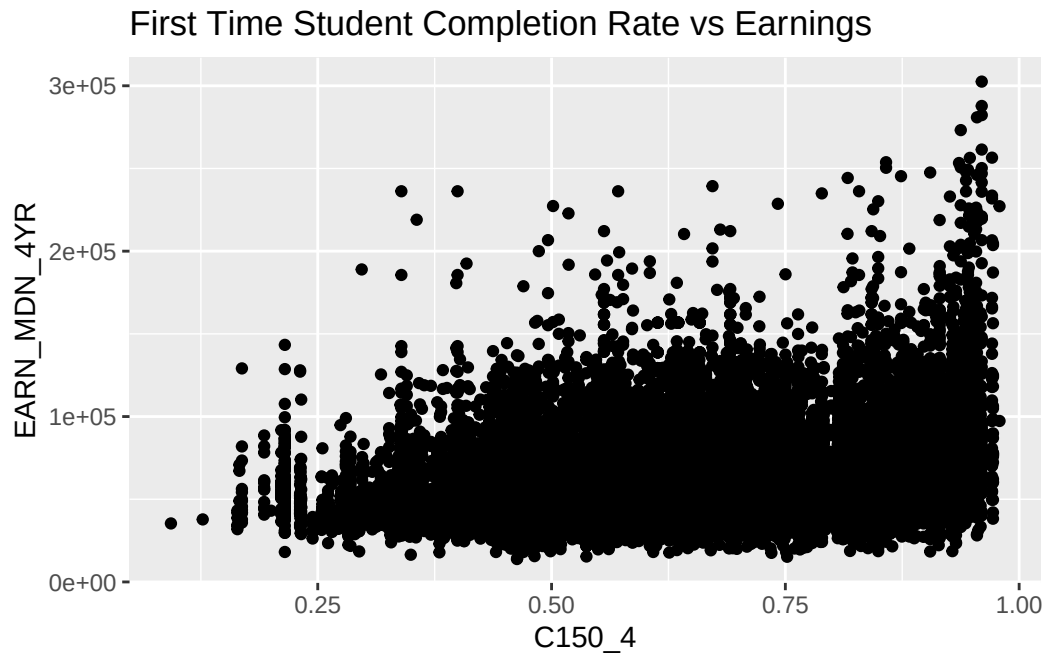


```
new_df <- df
new_df %>%
  # 1. Group the data and calculate the mean for each group
  group_by(CREDESC) %>%
  summarize(mean_value = mean(EARN_MDN_4YR, na.rm = TRUE)) %>%

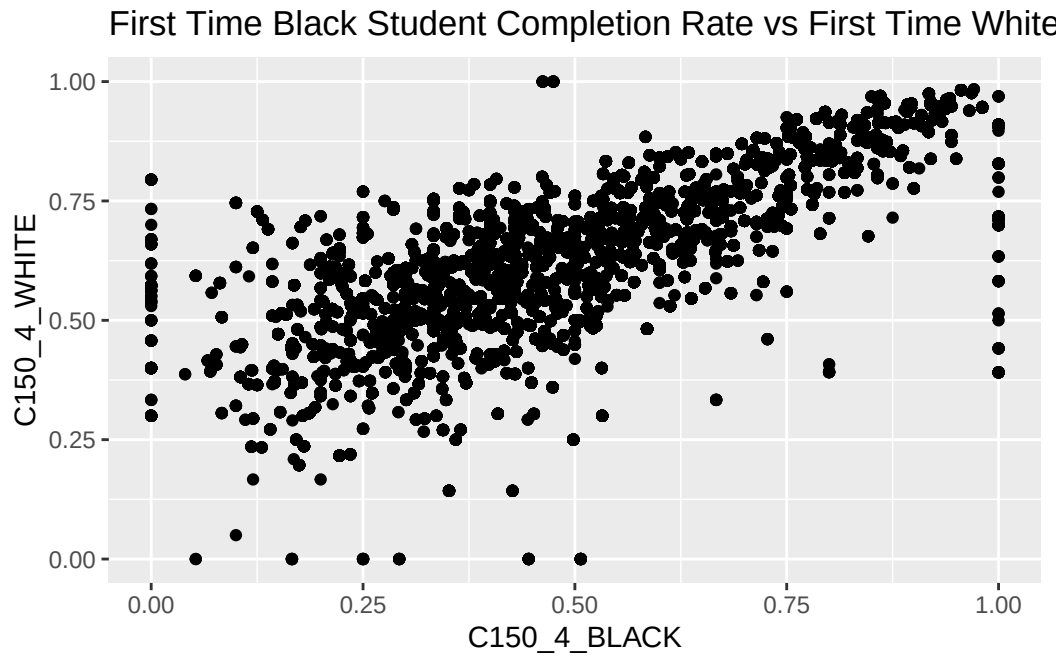
  # 2. Pass the aggregated data directly into ggplot
  ggplot(aes(x = CREDESC, y = mean_value, fill = CREDESC)) +
  geom_col() + # geom_col() plots the actual values provided in y
  labs(
    title = "Mean Value by Group",
    x = "Degree",
    y = "Average Mdn Earnings by Group"
  ) +
  theme_minimal()
```



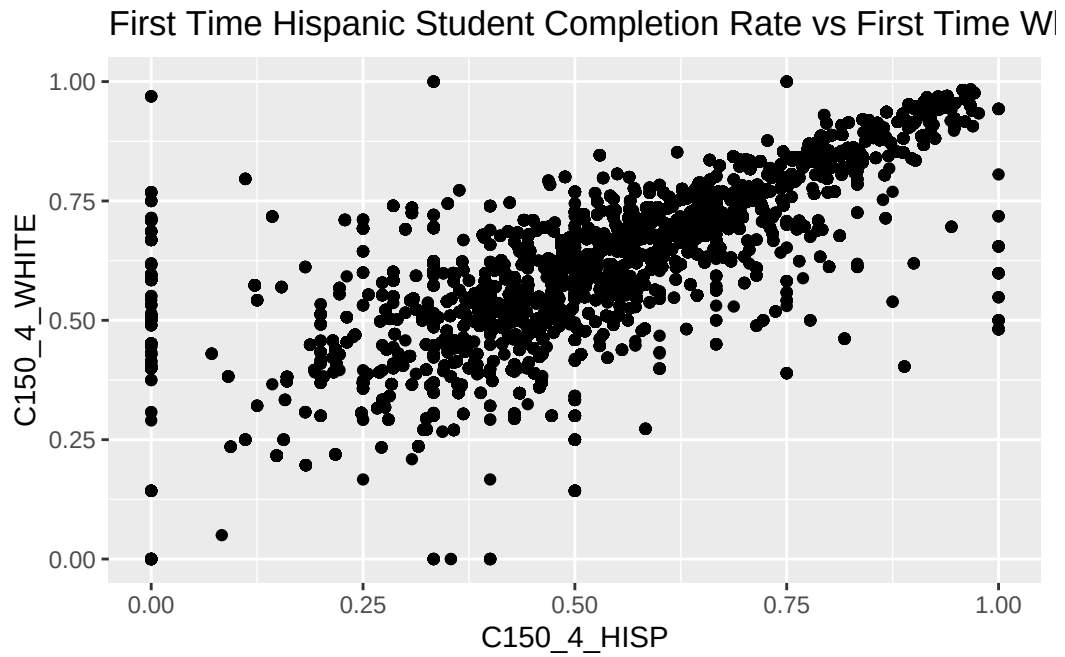
```
ggplot(df, aes(x=C150_4 , y = EARN_MDN_4YR)) +
  geom_point()+
  labs(
    title = "First Time Student Completion Rate vs Earnings"
  )
```



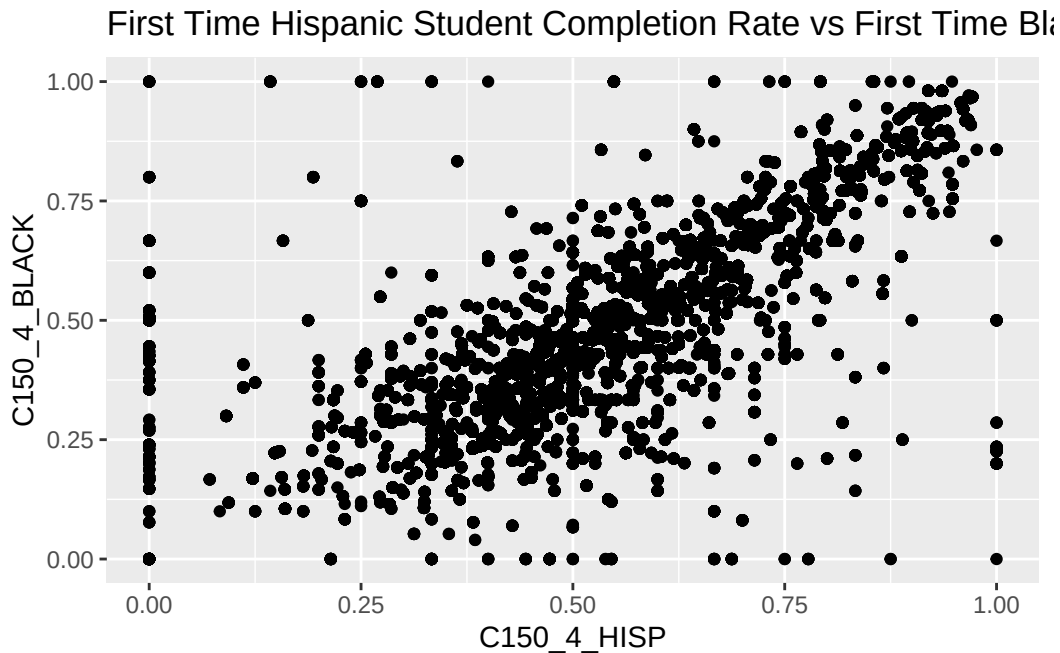
```
ggplot(df, aes(x=C150_4_BLACK , y = C150_4_WHITE)) +  
  geom_point()+  
  labs(  
    title = "First Time Black Student Completion Rate vs First Time White Student Completion  
  )
```



```
ggplot(df, aes(x=C150_4_HISP , y = C150_4_WHITE)) +  
  geom_point()+  
  labs(  
    title = "First Time Hispanic Student Completion Rate vs First Time White Student Complet.  
  )
```



```
ggplot(df, aes(x=C150_4_HISP , y = C150_4_BLACK)) +  
  geom_point()+  
  labs(  
    title = "First Time Hispanic Student Completion Rate vs First Time Black Student Complet.  
  )
```



TODO

Convert the following to numeric

One thing we ran into when using the data from individuals is that many of them have private information. This was recorded as **PS**, rather than a number recorded. So for many predictors, such as debt, we have to decide how to deal with them. One way is to impute the data. We could impute using mean imputation, where we set every **PS** values as the mean, either mean of everyone or mean of group, or we could predict what they are based off of other values, probably using a prebuilt function. We could also set them all to zero. We could also drop all the **PS** values.

Ruling out drop PS

When we look at the data, some of the predictors with **PS** can contain up to 90 percent of datapoints containing **PS**. If we through all of those points out, we would lose most of our data. Thus dropping all observations with **PS** would work for at most 1 variable, with only 50 of observations recorded as **PS**.

Selecting few of these

Due to the difficulties in working with much of this data, we are only going to work with 2-4 of these predictors.

```
DEATH_YR8_RT, COMP_ORIG_YR8_RT, COMP_4YR_TRANS_YR8_RT, PELL_COMP_4YR_TRA  
PELL_WDRAW_ORIG_YR8_RT, NOPELL_WDRAW_ORIG_YR8_RT, NOT1STGEN_COMP_4YR_T  
FIRSTGEN_COMP_ORIG_YR2_RT, OVERALL_YR2_N, DEBT_MDN, DEBT_ALL_STGP_ANY_ME  
BBRR1_FED_UG_DFLT
```

```
#str(df)
```